## DOCUMENTACIÓN DE ALQUILER DE MOTOS

### 1. ¿Para qué se hace este Código?

Este código se desarrolla para resolver una necesidad operativa del mundo real: **automatizar, organizar y gestionar el proceso de alquiler de vehículos** (específicamente motocicletas) dentro de una empresa o negocio.

A nivel conceptual y técnico, el código se realiza para cumplir con los siguientes objetivos fundamentales:

1. **Automatización de Cálculos Financieros:** Elimina la necesidad de realizar operaciones manuales para determinar la tarifa total que debe pagar un cliente. El programa toma automáticamente el costo por hora del vehículo y lo multiplica por el tiempo contratado, reduciendo errores humanos.
2. **Control y Registro de Información:** Permite vincular y asociar de manera organizada la información de un cliente (nombre y licencia) con los datos del vehículo asignado (placa y modelo), manteniendo trazabilidad de qué persona tiene alquilada cada motocicleta.
3. **Aplicación de Buenas Prácticas de Software (POO):** Demuestra la implementación práctica de los pilares de la Programación Orientada a Objetos en Java: Encapsulamiento (protección de datos con atributos privados y protegidos), Asociación (relación entre clases), Polimorfismo (uso de interfaces y métodos abstractos) y **Herencia** (implementación de una Clase Padre e Hija).
4. **Estandarización del Negocio con Interfaces:** Se crea la interfaz 'Calculable' para establecer una regla obligatoria en el software: cualquier servicio que cobre un valor debe implementar un método estándar para calcular su total. Esto facilita que en el futuro el sistema se pueda expandir a alquiler de carros, bicicletas o accesorios sin rehacer el programa.

### ESTRUCTURA DE HERENCIA INCORPORADA

- **Clase Padre (Superclase Abstracta):** Moto (abstract class Moto). Contiene los atributos comunes (placa, modelo, precioPorHora) y declara el método abstracto getTipoMoto().
- **Clase Hija (Subclase):** MotoDeportiva (class MotoDeportiva extends Moto). Hereda la estructura base de la clase padre mediante super(), añade su propio atributo específico (cilindrada) y desarrolla la lógica del método getTipoMoto().

### 2. Explicación Detallada de cada Componente

### A. La Interfaz 'Calculable'

Esta estructura actúa como un contrato o plantilla obligatoria en el sistema. Al ser una interfaz, no contiene código de ejecución interno, sino que declara la existencia del método abstracto calcularTotal(). Su función principal es obligar a cualquier clase que la implemente a calcular un valor financiero, garantizando que el sistema mantenga una estructura estandarizada y polimórfica.

**B. La Clase 'Cliente'**

Esta clase se encarga de modelar a la persona que solicita el servicio de alquiler. Contiene dos atributos privados para proteger la información: el nombre del usuario y su número de licencia de conducción. Cuenta con un constructor que recibe estos dos valores al momento de instanciar la clase para inicializarlos. Asimismo, incluye los métodos de acceso y modificación tradicionales (getters y setters) y el método obtenerDatos() que une y retorna el nombre junto con la licencia en una sola cadena de texto formateada.

**C. La Clase Padre Abstracta 'Moto'**

Representa la abstracción física base de la motocicleta en la flota de la empresa. Se define como una clase abstracta (abstract class Moto) para impedir que se creen instancias genéricas sin especificar su categoría. Protege la información de la placa y el modelo mediante atributos privados, y declara el atributo de tarifa con alcance protegido (protected double precioPorHora) para permitir que las clases hijas puedan acceder a él directamente. Incluye sus respectivos getters y setters, así como la firma del método abstracto public abstract String getTipoMoto();, el cual obliga a cualquier subclase a definir la categoría del vehículo.

**D. La Clase Hija 'MotoDeportiva'**

Esta clase representa una especialización de vehículo y hereda directamente de la clase padre mediante la instrucción extends Moto. Incorpora un atributo propio exclusivo de esta categoría (private int cilindrada) y utiliza la llamada super(placa, modelo, precioPorHora) dentro de su constructor para reutilizar la inicialización definida en el padre. Implementa obligatoriamente el método getTipoMoto(), retornando una descripción personalizada que incluye la cilindrada del vehículo.

**E. La Clase 'Alquiler'**

Esta clase funciona como el núcleo transaccional del sistema, ya que se encarga de relacionar un objeto de tipo 'Cliente' con un objeto de tipo 'Moto' (pudiendo recibir cualquier clase hija como MotoDeportiva gracias al polimorfismo) durante una cantidad específica de horas. Al incorporar la instrucción implements Calculable, la clase asume un compromiso formal de desarrollar el método calcularTotal(). En su lógica interna, este método toma el precio por hora de la moto asignada y lo multiplica por la cantidad de horas registradas.

**F. La Clase Principal 'Alquiler De Motos'**

Es la clase ejecutable del sistema que contiene el método principal main. Su funcionamiento consiste en poner en marcha el flujo de la aplicación mediante los siguientes pasos secuenciales:

1. **Instanciación de la Moto Hija:** Se crea un objeto de la subclase MotoDeportiva pasando como argumentos su placa, modelo, precio por hora y cilindrada.
2. **Instanciación del Cliente:** Se crea un objeto de la clase Cliente proporcionando el nombre de la persona y su respectivo número de licencia.
3. **Creación del Alquiler (Polimorfismo):** Se instancia la clase Alquiler vinculando al cliente y a la moto por un número determinado de horas. Esta instancia se almacena utilizando el tipo de la interfaz 'Calculable', lo que demuestra el uso de polimorfismo en Java.
4. **Salida de Resultados por Consola:** Finalmente, el programa imprime en la consola un resumen detallado que incluye los datos del cliente, la categoría y especificaciones de la moto, y el valor total calculado.

**3. Conclusión de la Arquitectura**

La integración de la Herencia (mediante la clase padre abstracta Moto y la clase hija MotoDeportiva), junto con la interfaz estandarizada Calculable y la clase transaccional Alquiler, consolida un diseño de software limpio, modular, fácil de mantener y preparado para futuras ampliaciones del negocio (como añadir otras clases hijas como MotoScooter o MotoTouring).